

Competitive Programming

Section 45 Editorial: Flows and Matchings

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Max Flow	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
2.7	Tests that matter	5
3	B. Bipartite Matching	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
3.7	Tests that matter	7
4	C. Min Cut Reachable Side	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
4.7	Tests that matter	9
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Maximum Flow. Dinic's algorithm works on the residual graph. A BFS builds levels using only edges with positive residual capacity. A DFS then sends flow only from level d to level $d+1$, which forms a blocking flow for the current level graph. When the sink is unreachable in BFS, there is no augmenting path left. By the max-flow min-cut theorem, the current flow is maximum. The reverse edge stores the option to cancel earlier choices, which is essential for correctness.

B. Bipartite Matching. Model each left vertex as choosing at most one right vertex. The DFS tries to match a left vertex directly, and if the right vertex is already used, it recursively asks the previous owner to move elsewhere. A successful recursive move creates an augmenting path and increases the matching size by one. When no DFS from any unmatched left vertex succeeds, no augmenting path remains, so the matching is maximum.

C. Minimum Cut After Flow. After computing max flow, run a reachability search from the source in the final residual graph. Let S be the reachable set. No residual edge of positive capacity crosses from S to $V \setminus S$; otherwise the endpoint would be reachable. Thus all original capacity across that cut is saturated, and its value equals the max flow. The printed cut is therefore a minimum cut.

Common pitfalls. Always update reverse edges together with forward edges. For matching, reset the visited array for each DFS root. For min cut, use the residual graph after max flow, not the original graph.

The three exercises are different views of augmenting paths.

- A implements Dinic. BFS builds a residual level graph; DFS sends blocking flow along increasing levels; reverse edges allow rerouting earlier choices.
- B solves bipartite matching with augmenting DFS. A right vertex can be reassigned if its current left partner can find another route.
- C runs max flow, then explores the residual graph from the source. The reachable set is the source side of a minimum cut.

For max-flow correctness, the key terminal condition is: if the sink is not reachable in the residual graph, there is no augmenting path, so the current flow is maximum.

2 A. Max Flow

2.1 Restatement

Given a directed capacitated graph from node 1 to node n, print the maximum flow.

2.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

2.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

2.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>>g;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Di
```

2.6 Full Python solution

```

import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
    flow=0
    while True:
        level=[-1]*n; level[s]=0; dq=deque([s])
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
        if level[t]<0: break
        it=[0]*n
        def dfs(u,f):
            if u==t: return f
            for i in range(it[u],len(g[u])):
                it[u]=i; v,c,rev=g[u][i]
                if c and level[v]==level[u]+1:
                    ret=dfs(v,min(f,c))
                    if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
            return 0
        while True:
            pushed=dfs(s,10**18)
            if not pushed: break
            flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    print(dinic(n,edges,0,n-1)[0])
if __name__=="__main__": main()

```

2.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

3 B. Bipartite Matching

3.1 Restatement

Given edges between left and right parts, print the maximum matching size.

3.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

3.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

3.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

3.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
```

```

match=[-1]*m
def dfs(u,vis):
    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()

```

3.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

4 C. Min Cut Reachable Side

4.1 Restatement

After max flow from 1 to n, print the flow value and the source-side reachable vertices in the residual graph.

4.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

4.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

4.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>g
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Dinic
```

4.6 Full Python solution

```

import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
    flow=0
    while True:
        level=[-1]*n; level[s]=0; dq=deque([s])
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
        if level[t]<0: break
        it=[0]*n
        def dfs(u,f):
            if u==t: return f
            for i in range(it[u],len(g[u])):
                it[u]=i; v,c,rev=g[u][i]
                if c and level[v]==level[u]+1:
                    ret=dfs(v,min(f,c))
                    if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
            return 0
        while True:
            pushed=dfs(s,10**18)
            if not pushed: break
            flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    flow,seen=dinic(n,edges,0,n-1,True); print(flow); print(' '.join(str(i+1) for i,x in enumerate(seen) if x))
if __name__=="__main__": main()

```

4.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.